ESP32-CAM Face Recognition System

Project Documentation & Interview Preparation Guide

Munazza Hawaldar — Diploma in Electronics & Telecommunication Engineering

September 2026

Part A: Project Documentation
- Project Overview
- Features
- Problem Statement
- Objectives
- System Architecture
- Hardware Requirements
- Software Requirements
- Technologies Used
- Repository Structure
- Installation & Setup
- ESP32-CAM Configuration
- Wi-Fi Configuration
- Python Environment Setup
- Training Data Structure
- How Face Detection Works
- How LBPH Face Recognition Works
- How the Flask Dashboard Works
- How to Run the Project
- Troubleshooting
- Limitations
- Future Enhancements
- Safety & Privacy Considerations

Part B: Interview Preparation Guide
- 1. 30-Second Explanation
- 2. 1-Minute Technical Explanation
- 3. Explanation of Every File
- 4. ESP32-CAM Code Explanation
- 5. Python/OpenCV Code Explanation
- 6. Explanation of Haar Cascade
- 7. Explanation of LBPH
- 8. Detection vs Recognition
- 9. Flask’s Role
- 10. Common Interview Questions & Answers
- 11. Possible Limitations
- 12. Possible Future Improvements
- 13. Resume/GitHub Keywords

Part A: Project Documentation

A diploma-level embedded systems + IoT + computer vision prototype that streams live video from an AI Thinker ESP32-CAM module over Wi-Fi to a Python application, which performs real-time **face detection** (Haar Cascade) and **face recognition** (LBPH), and displays the result on a simple **Flask web dashboard**.

> **Student Prototype Notice:** This project was built as a diploma-level Electronics & Telecommunication Engineering project to demonstrate the integration of embedded hardware, networking, and computer vision. It is **not** a production-grade or security-certified system, and no specific recognition accuracy is claimed unless it has actually been measured on your own test data.

Project Overview

The system uses an **AI Thinker ESP32-CAM** module to capture live video and transmit it over Wi-Fi as an MJPEG stream. A **Python** application running on a laptop/PC connects to this stream, uses **OpenCV** to detect faces in each frame with a **Haar Cascade classifier**, and then identifies known individuals using an **LBPH (Local Binary Patterns Histograms) face recognizer** trained on a small custom dataset. The processed video (with bounding boxes and name labels) is served to a browser through a **Flask** web dashboard.

Features

- Live video streaming from ESP32-CAM over Wi-Fi (MJPEG).
- Real-time face **detection** using OpenCV Haar Cascade.
- Face **recognition** (Known / Unknown) using LBPH.
- Bounding boxes and name labels drawn on the live feed.
- Simple browser-based dashboard (Flask + HTML/CSS) to view the stream.
- One-click model retraining from the dashboard.
- Fallback to a laptop webcam for development/testing without hardware.
- Modular, well-commented codebase suitable for demonstration in an interview or viva.

Problem Statement

Many introductory embedded-systems and IoT courses teach camera modules and Wi-Fi connectivity, and many computer-vision courses teach face detection/recognition — but few small projects connect the two into a single working pipeline: capturing images with low-cost embedded hardware and processing them with computer vision on a separate compute device. This project demonstrates that end-to-end pipeline using affordable, widely available components (ESP32-CAM) and well-established, beginner-friendly computer vision techniques (Haar Cascade + LBPH), without requiring a GPU or deep-learning framework.

Objectives

- Interface an ESP32-CAM module and stream video over a local Wi-Fi network.
- Receive and decode the video stream in Python using OpenCV.
- Detect human faces in real time using a Haar Cascade classifier.
- Train a lightweight face recognizer (LBPH) on a small custom dataset.
- Classify detected faces as a known individual or “Unknown”.
- Present the results through a live, browser-accessible dashboard.
- Keep the entire pipeline understandable and explainable for a diploma-level fresher (no black-box deep learning).

System Architecture

Data flow: ESP32-CAM capture -> Wi-Fi stream -> Python/OpenCV frame processing -> Haar Cascade face detection -> LBPH face recognition -> Known/Unknown result -> bounding box + label drawn on frame -> re-encoded as JPEG -> streamed to the Flask web dashboard -> rendered in the browser.

The ESP32-CAM exposes a `/stream` endpoint over Wi-Fi. The Python application (`face_engine.py`) pulls frames from this endpoint, detects and recognizes faces, and hands annotated frames to the Flask app (`app.py`), which exposes `/video_feed` (MJPEG), `/status` (JSON) and `/train` (POST) routes consumed by the dashboard (`web/index.html`).

Hardware Requirements

Component	Purpose
AI Thinker ESP32-CAM module	Captures and streams video
USB-to-TTL (FTDI) programmer	Flashes the Arduino sketch onto the ESP32-CAM (it has no onboard USB port)
Computer / laptop	Runs the Python/OpenCV/Flask application
Wi-Fi network (2.4 GHz)	Connects the ESP32-CAM and the computer on the same LAN
Jumper wires	Wiring the programmer to the ESP32-CAM during flashing
5V power supply (via USB or external)	Powers the ESP32-CAM during operation

Software Requirements

- Arduino IDE (with ESP32 board support installed)
- Python 3.9 or newer
- pip (Python package manager)
- A code editor (VS Code, etc.) — optional but recommended
- Git (to clone/push this repository)

Technologies Used

- **ESP32-CAM (AI Thinker)** — embedded camera + Wi-Fi module
- **Arduino IDE / C++** — firmware for camera capture and streaming
- **Python** — application logic, orchestration
- **OpenCV** — image processing, Haar Cascade, LBPH
- **Haar Cascade** — classical face detection algorithm
- **LBPH Face Recognizer** — classical face recognition algorithm
- **Flask** — lightweight Python web framework for the dashboard
- **HTML/CSS** — dashboard front-end
- **NumPy** — numerical array handling for image data
- **Wi-Fi (802.11)** — transport between ESP32-CAM and PC
- **Git/GitHub** — version control and project hosting

Repository Structure

```
ESP32-CAM-Face-Recognition/
│
├── esp32/
│   └── esp32_cam_stream.ino     # Arduino firmware: camera init + Wi-Fi MJPEG server
│
├── python/
```

```
│   ├── app.py                  # Flask app: dashboard + video route + train endpoint
│   ├── config.py               # All tunable settings (IP, thresholds, paths)
│   ├── face_engine.py          # Face detection (Haar) + recognition (LBPH) logic
│   └── requirements.txt        # Python dependencies
│
├── data/
│   └── faces/
│       └── README.md           # Explains training data folder structure
│
├── models/
│   └── README.md               # Explains how to get haarcascade.xml + generated files
│
├── web/
│   └── index.html               # Dashboard front-end (video feed + status panel)
│
├── .gitignore
└── README.md                    # Full documentation
```

Installation & Setup

```
# 1. Clone the repository
git clone https://github.com/<your-username>/ESP32-CAM-Face-Recognition.git
cd ESP32-CAM-Face-Recognition

# 2. Set up the Python environment (see "Python Environment Setup" below)
cd python
python -m venv venv
source venv/bin/activate      # On Windows: venv\Scripts\activate
pip install -r requirements.txt

# 3. Add the Haar Cascade file to models/ (see models/README.md)

# 4. Flash esp32/esp32_cam_stream.ino to your ESP32-CAM (see below)

# 5. Add training photos to data/faces/<PersonName>/ (see below)

# 6. Run the app
python app.py
```

ESP32-CAM Configuration

1. Open `esp32/esp32_cam_stream.ino` in the Arduino IDE.
2. Install ESP32 board support (Boards Manager -> search “esp32”).
3. Select **Board: AI Thinker ESP32-CAM** (or “ESP32 Wrover Module” with matching pin settings).
4. Connect the ESP32-CAM to your USB-to-TTL programmer:
   - `5V` -> `5V`, `GND` -> `GND`, `U0R` -> `TX`, `U0T` -> `RX`
   - Connect `GPIO0` to `GND` to enable flashing mode, then reset the board.
5. Update `WIFI_SSID` and `WIFI_PASSWORD` in the sketch (see next section).
6. Upload the sketch. Open the Serial Monitor (115200 baud) to see the assigned IP address once it connects to Wi-Fi.
7. Disconnect `GPIO0` from `GND` and reset the board to run normally.

Wi-Fi Configuration

- The ESP32-CAM and the computer running the Python app **must be on the same Wi-Fi network** (same LAN/subnet).
- Edit these two lines near the top of `esp32/esp32_cam_stream.ino`:

```
const char *WIFI_SSID     = "YOUR_WIFI_SSID";
```

```
const char *WIFI_PASSWORD = "YOUR_WIFI_PASSWORD";
```

- **Never commit real Wi-Fi credentials to GitHub.** For a shared/public repo, consider moving these into a separate `secrets.h` file that is listed in `.gitignore` (already prepared for this).
- After flashing, note the IP address printed in the Serial Monitor and update `ESP32_CAM_IP` in `python/config.py` accordingly.

Python Environment Setup

```
cd python
python -m venv venv
source venv/bin/activate        # Windows: venv\Scripts\activate
pip install -r requirements.txt
```

Key packages installed: - `opencv-contrib-python` — OpenCV plus the `cv2.face` module needed for LBPH (the regular `opencv-python` package does NOT include this). - `numpy` — array/image data handling. - `Flask` — the web dashboard server. - `requests` — used to pull the MJPEG stream from the ESP32-CAM.

Training Data Structure

```
data/faces/
├── Person1/
│   ├── 01.jpg
│   ├── 02.jpg
│   └── 03.jpg
└── Person2/
    ├── 01.jpg
    └── 02.jpg
```

- One folder per person; the **folder name is used as the recognized label**.
- 5–20 images per person is a reasonable starting point for a student prototype, with varied lighting/angles.
- No real face images are included in the repository — see `data/faces/README.md` for details and privacy notes.
- After adding images, train the model either by clicking **“Retrain Model”** on the dashboard, calling `POST /train`, or running `FaceEngine().train()` directly in a Python shell.

How Face Detection Works

Haar Cascade is a classical, pre-deep-learning object detection method (Viola-Jones algorithm). It works by:

1. Sliding rectangular “features” (patterns of light/dark regions, like eyes being darker than cheeks) across the image at multiple scales.
2. Using a cascade of simple classifiers trained on thousands of positive (face) and negative (non-face) example images.
3. Quickly rejecting regions that clearly aren’t faces early in the cascade, and only running deeper checks on promising regions — this makes it fast enough to run in real time on a normal laptop CPU.
4. Returning bounding boxes `(x, y, width, height)` for each detected face region.

In this project, `face_engine.py`’s `detect_faces()` method wraps OpenCV’s `CascadeClassifier.detectMultiScale()`, using the pre-trained `haarcascade_frontalface_default.xml` model that ships with OpenCV.

How LBPH Face Recognition Works

LBPH (Local Binary Patterns Histograms) is a classical face *recognition* algorithm (distinct from detection):

1. For each pixel in a face image, it compares the pixel's brightness to its 8 neighbours, encoding the result as a binary pattern (an 8-bit "Local Binary Pattern").
2. The face image is divided into a grid of regions, and a histogram of these binary patterns is computed for each region.
3. All the regional histograms are concatenated into a single feature vector representing that face.
4. During recognition, the histogram of a new face is compared to all stored histograms using a distance metric (e.g. chi-square distance).
5. The closest match's label is returned, along with a **confidence score that is actually a distance** — lower means a closer/better match, not a percentage.

In this project, `LBPH_CONFIDENCE_THRESHOLD` (in `config.py`) decides how close a match must be before it's labelled with a person's name instead of "Unknown". This threshold is **not a measured accuracy** — it must be tuned by testing with your own camera and lighting.

How the Flask Dashboard Works

- `app.py` creates a single shared `FaceEngine` instance at startup (loading the Haar Cascade and any previously trained LBPH model).
- The `/video_feed` route is a generator-based Flask `Response` that continuously: pulls a frame from the ESP32-CAM (or webcam) -> processes it (detect + recognize + annotate) -> re-encodes it as JPEG -> yields it in `multipart/x-mixed-replace` format.
- The browser's `<img>` tag on `web/index.html` points directly at `/video_feed`, so it renders each new JPEG frame as it arrives, producing a live video effect without any custom JavaScript video player.
- `/status` returns JSON describing whether a model is trained and which people are known, polled every few seconds by the dashboard.
- `/train` (POST) re-runs training on the current contents of `data/faces/` and reloads the model.

How to Run the Project

1. Make sure the ESP32-CAM is powered on and connected to Wi-Fi (or set `USE_LOCAL_WEBCAM = True` in `config.py` to test with a laptop webcam instead).
2. Confirm `ESP32_CAM_IP` in `python/config.py` matches the IP shown in the Arduino Serial Monitor.
3. Add at least one person's training photos under `data/faces/`.
4. From the `python/` folder, run:

```
python app.py
```

5. Open `http://localhost:5000` in your browser.
6. Click **"Retrain Model"** on the dashboard (first-time setup) to train on your current photos.
7. Watch the live annotated video feed — detected faces will be boxed in green (recognized) or red (unknown).

Troubleshooting

Problem	Likely Cause / Fix
`ModuleNotFoundError: cv2.face`	You installed `opencv-python` instead of `opencv-contrib-python`. Uninstall both and reinstall only `opencv-contrib-python`.

Blank/black video feed	Check `ESP32_CAM_IP` is correct and the ESP32-CAM and PC are on the same Wi-Fi network. Try opening `http://<ESP32_IP>/stream` directly in a browser.
`ConnectionError: Could not connect to ESP32-CAM`	The device may not be powered on, may have failed to join Wi-Fi, or the IP address changed (many routers assign IPs dynamically — consider a static IP/DHCP reservation).
Camera init failed on ESP32-CAM	Check wiring/power — the ESP32-CAM needs a stable 5V supply; USB port power from a laptop is sometimes insufficient during Wi-Fi transmission.
No faces detected	Ensure good, even lighting and that the face is reasonably front-facing and large enough in frame; check `HAAR_MIN_FACE_SIZE` in `config.py`.
Everyone recognized as “Unknown”	Retrain the model after adding photos; try relaxing `LBPH_CONFIDENCE_THRESHOLD` slightly in `config.py`.
Flashing the ESP32-CAM fails	Confirm `GPIO0` is connected to `GND` before uploading, and disconnected before running normally; check TX/RX wiring isn’t swapped.
Stream is laggy/slow	Lower the frame resolution (`FRAMESIZE_QVGA` instead of `VGA`) or increase the `delay()` in the ESP32 sketch to reduce frame rate.

Limitations

- Haar Cascade can produce false positives/negatives, especially with non-frontal faces, poor lighting, or partial occlusion (masks, glasses glare, etc.).
- LBPH is a classical (non-deep-learning) method — it generally requires good, consistent training images and works best with a small number of known people; it does not scale as well as modern deep-learning face recognition for large populations.
- No formally measured accuracy is claimed in this repository — actual performance depends heavily on camera quality, lighting, distance, and training data quality.
- The ESP32-CAM’s MJPEG stream over Wi-Fi can be affected by network congestion, resulting in dropped or delayed frames.
- This is a **prototype**, not a hardened or production-ready system — it lacks authentication on the dashboard, encrypted video transport, and has not been security-audited.

Future Enhancements

- Add HTTPS and basic authentication to the Flask dashboard.
- Store recognition events/logs (timestamp + name) for later review.
- Add an on-device (ESP32) or server-side alert (e.g. email/notification) when an unknown face is detected.
- Explore deep-learning-based face embeddings (e.g. FaceNet/dlib) for improved accuracy, as a comparison against LBPH.
- Add a simple UI to capture new training photos directly from the live stream instead of manually placing files.
- Package the Python app with Docker for easier setup.
- Measure and report actual detection/recognition accuracy on a labelled test set.

Safety & Privacy Considerations

- This project processes **biometric data (faces)** — treat any training images and the trained model as sensitive personal data.
- Do **not** upload real people’s photographs, the trained `lbph_model.yml`, or `labels.json` to a public GitHub

repository (all are excluded via `.gitignore` by default).
- Obtain **explicit consent** from anyone whose face you use for training or testing, and inform them how their image will be used/stored.
- This system has **not** been evaluated for fairness/bias across different demographics, and should not be relied upon for security, access control, or any decision with real-world consequences.
- Always disclose clearly that this is a student learning project, not a certified or production biometric system.

Part B: Interview Preparation Guide

1. 30-Second Explanation

“I built a face recognition system using an ESP32-CAM module that streams live video over Wi-Fi to my laptop. On the Python side, I use OpenCV to detect faces with a Haar Cascade classifier, then recognize known people using an LBPH face recognizer trained on a small photo dataset. The result — a bounding box with the person’s name or ‘Unknown’ — is shown live on a Flask web dashboard in the browser.”

2. 1-Minute Technical Explanation

“The ESP32-CAM, running a custom Arduino sketch, initializes the OV2640 camera and connects to Wi-Fi. It runs a lightweight HTTP server that serves an MJPEG stream — basically a continuous sequence of JPEG frames — at a `/stream` endpoint. On my laptop, a Python script pulls that stream using the `requests` library, manually parsing out JPEG frame boundaries, and decodes each frame with OpenCV. Each frame is converted to grayscale and passed to a Haar Cascade classifier, which detects face regions using pattern-based feature matching. Each detected face is cropped, resized, and passed to an LBPH recognizer, which was pre-trained on labeled images stored per-person in a `data/faces/` folder. LBPH compares the new face’s local binary pattern histogram against stored ones and returns the closest match plus a distance score, which I threshold to decide Known vs Unknown. Finally, I draw the bounding box and label on the frame, re-encode it as JPEG, and stream it into a Flask app, which serves it to a simple HTML dashboard using `multipart/x-mixed-replace`, the same technique old-school IP cameras use for live video in a browser.”

3. Explanation of Every File

File	Purpose
`esp32/esp32_cam_stream.ino`	Arduino firmware: initializes camera, connects to Wi-Fi, serves MJPEG stream
`python/config.py`	Central place for all settings — IP address, thresholds, file paths
`python/face_engine.py`	Core CV logic — detection, training, recognition, frame source handling
`python/app.py`	Flask server — routes for dashboard, video stream, status, training
`python/requirements.txt`	Pinned Python dependencies
`web/index.html`	Browser dashboard — video `<img>` tag + status panel + retrain button
`data/faces/`	Per-person folders of training images (empty in repo, gitignored)
`models/`	Holds the Haar Cascade XML + generated LBPH model + labels (gitignored except README)

`.gitignore`	Keeps secrets, private photos, and generated model files out of GitHub
`README.md`	Full documentation

4. ESP32-CAM Code Explanation

The pin map at the top matches the AI Thinker board's fixed camera wiring — these can't be changed. `initCamera()` configures resolution/quality (higher if PSRAM is present, lower otherwise, to avoid crashes) and calls `esp_camera_init()`. `connectToWiFi()` connects with a timeout and auto-restarts if it fails. Instead of pulling in a heavy web server library, a minimal HTTP server is written by hand using `WiFiServer`: it reads the incoming request, and if it's `GET /stream`, calls `streamMJPEG()`, which loops forever grabbing a JPEG frame from the camera buffer and writing it to the client wrapped in `multipart/x-mixed-replace` boundaries — that's what lets a browser or OpenCV treat it as continuous video.

5. Python/OpenCV Code Explanation

`face_engine.py` has a `FaceEngine` class that loads the Haar Cascade once at startup (not per-frame, for performance) and creates a `cv2.face.LBPHFaceRecognizer_create()` instance. `train()` walks the `data/faces/` folders, detects the face in each training photo (so background is excluded), resizes it consistently, and feeds all of them into `recognizer.train()`. `process_frame()` is the live pipeline: convert to grayscale -> `detectMultiScale()` for face boxes -> `recognizer.predict()` for each face -> draw a green box if confident, red if not. `frame_generator()` abstracts the video source so `app.py` doesn't care whether frames come from the ESP32 stream or a local webcam fallback.

6. Explanation of Haar Cascade

A classical (pre-deep-learning) object detector from Viola-Jones (2001). It uses simple rectangular features — like "the eye region is darker than the cheek region below it" — and combines thousands of these into a cascade of increasingly strict classifiers. Most non-face regions get rejected in the first few cheap stages, which is why it's fast enough for real-time use on an ordinary CPU. It only detects *that* a face is present and *where* — it has no idea *whose* face it is.

7. Explanation of LBPH

LBPH looks at each pixel, compares it to its 8 neighbors (brighter or darker), and encodes that as an 8-bit binary number — the Local Binary Pattern. The face image is split into a grid, a histogram of these patterns is built per grid cell, and all histograms are concatenated into one feature vector representing that face. To recognize a new face, OpenCV compares its histogram to every stored histogram using a distance metric and returns the closest match plus a distance score (lower = better match — it's not a percentage). It's called "local" because each small patch contributes independently, which makes it fairly tolerant to lighting changes.

8. Detection vs Recognition

- **Detection** answers "Is there a face here, and where?" — Haar Cascade's job.
- **Recognition** answers "Whose face is this?" — LBPH's job, and it only runs on regions detection already found.

9. Flask's Role

Flask is the glue between the CV pipeline and the browser. It doesn't do any image processing itself — it just: (a) serves the HTML dashboard, (b) exposes `/video_feed`, a generator-based route that continuously yields annotated JPEG frames in the MJPEG format browsers understand natively via an `<img>` tag, (c) exposes `/status` for JSON info the dashboard polls, and (d) exposes `/train` so retraining can be triggered with a button click instead of restarting the app.

10. Common Interview Questions & Answers

Q: Why Haar Cascade instead of a deep-learning detector like MTCNN or YOLO? A: It's lightweight, needs no GPU, ships built into OpenCV, and is easy to explain and reason about — appropriate for a diploma-level prototype. DNN-based detectors are a natural future improvement.

Q: Why LBPH instead of modern embeddings (FaceNet, ArcFace)? A: LBPH is simple, fast, works with very few training images per person, and doesn't need a pretrained deep model — good for a small, explainable student project.

Q: What does the "confidence" value from LBPH actually mean? A: It's a distance, not a probability — lower means more similar. It's thresholded to decide Known vs Unknown, and that threshold has to be tuned by testing, not assumed.

Q: How do you handle multiple faces in a frame? A: `detectMultiScale()` returns all face boxes in one frame; the code loops through each and runs recognition individually.

Q: What happens if the ESP32-CAM loses Wi-Fi connection? A: The Arduino side has a Wi-Fi timeout that reboots the device; on the Python side, `face_engine.py` raises a `ConnectionError` that `app.py` catches so the whole server doesn't crash.

Q: Why opencv-contrib-python and not opencv-python? A: The LBPH recognizer lives in `cv2.face`, which is only included in the contrib package.

Q: How would you improve accuracy? A: More/better training images, consistent lighting, tuning the confidence threshold, or upgrading to a deep-learning-based recognizer.

Q: Is this secure enough for real access control? A: No — it's a prototype without authentication, encryption, or measured accuracy; not meant for security-critical deployment.

11. Possible Limitations

False positives/negatives from Haar Cascade in poor lighting or angled faces; LBPH doesn't scale well to many people; no measured accuracy; Wi-Fi stream can lag; no dashboard authentication.

12. Possible Future Improvements

HTTPS + login for the dashboard, recognition event logging, alerts on unknown faces, deep-learning-based recognition for comparison, in-browser training photo capture, Docker packaging, formal accuracy benchmarking.

13. Resume/GitHub Keywords

`ESP32-CAM` · `Embedded Systems` · `IoT` · `Computer Vision` · `OpenCV` · `Haar Cascade` · `LBPH` · `Face Detection` · `Face Recognition` · `Python` · `Flask` · `REST API` · `MJPEG Streaming` · `Real-time Video Processing` · `Arduino/C++` · `Wi-Fi Networking` · `NumPy` · `Git/GitHub` · `Embedded-to-Cloud Pipeline`